



This repository Search

Pull requests Issues Gist



uwescience / datasci_course_materials

Watch

370

Star

667

Fork

2,240

<> Code

Issues 4

Pull requests 15

Wiki

Pulse

Graphs

Branch: master

datasci_course_materials / assignment4 / assignment4.md

Find file

Copy path



billhowe markdown formatting

7acac5e on Nov 23, 2015

1 contributor

176 lines (100 sloc) 12.8 KB

Raw

Blame

History



We cannot reimburse you for any charges

Terminating an AWS cluster

When you are done running Pig scripts, make sure to **ALSO** terminate your job flow. This is a step that you need to do *in addition to* * stopping pig and Hadoop (if necessary).

1. Go to the [Management Console](#).
2. Select the job in the list.
3. Click the Terminate button (you may also need to turn off Termination protection).
4. Wait for a while (may take minutes) and recheck until the job state becomes TERMINATED.

If you fail to terminate your job and only close the browser, or log off AWS, your AWS will continue to run, and AWS will continue to charge your credit card: for hours, days, and weeks. Make sure you don't leave the console until you have confirmation that the job is terminated.

Notes

This assignment will be very difficult from Windows; the instructions assume you have access to a Linux command line.

The quiz should cost no more than 5-10 dollars if you only use small aws instances

Problem 0: Setup your Pig Cluster

1. Follow [these instructions](#) to setup the cluster. NOTE: It will take you a good **60 minutes** to go through all these instructions without even trying to run example.pig at the end. But they are worth it. You are learning how to use the Amazon cloud, which is by far the most popular cloud platform today. At the end, the instructions will refer to *example.pig*. This is the name of the sample program that we will run in the next step.
2. You will find example.pig in the [course materials repo](#). example.pig is a Pig Latin script that loads and parses the billion triple dataset that we will use in this assignment into triples: (subject, predicate, object). Then it groups the triples by their object attribute and sorts them in descending order based on the count of tuple in each group.
3. Follow the README.txt: it provides more information on how to run the sample program called example.pig.
4. There is nothing to turn in for Problem 0

Useful Links

[Pig Latin reference](#)

[Counting rows in an alias](#)

Quiz Description

In this assignment, we will perform some basic analysis over a relatively large social network graph derived from the billion triple challenge associated with the Semantic Web. The dataset is represented using the Resource Description Framework (RDF), where each record represents one edge in the graph.

RDF data is represented as a set of triples of the form:

```
subject predicate object [context]
```

The [context] is not part of the triple, but is sometimes added to tell where the data is coming from. For example, file btc-2010-chunk-200 contains the two "triples" (they are actually "quads" because they have the context too):

```
<http://www.last.fm/user/ForgottenSound> <http://xmlns.com/foaf/0.1/nick> "ForgottenSound" <http://rdf.opiumfield.com  
<http://dblp.13s.de/d2r/resource/publications/journals/cg/WestermannH96> <http://xmlns.com/foaf/0.1/maker> <http://dt
```

The first says that Webpage <http://www.last.fm/user/ForgottenSound> has the nickname "ForgottenSound"; the second describes the maker of another webpage. foaf stands for *Friend of a Friend*. Confused? You don't need to know what they mean; some of the many triples refer to music, <http://dbtune.org>, others refer to company relationships, etc. For our purpose, these triples are just a large collection of triples. There were 317 2GB files in the [billion triple dataset](#) when we downloaded it. We uploaded them to Amazon's Web Services in S3: there were some errors, and only 251 uploaded correctly, for a total of about 550 GB of data.

This graph is similar in size to the [web graph](#). As part of this assignment, we will compute the out-degree of each node in the graph. The out-degree of a node is the number of edges coming out of the node. This is an important property. If a graph is random, the out-degree of nodes will follow an exponential distribution (i.e., the number of nodes with degree d should be $\exp(-c \cdot d)$ for some constant c). We will write the script in Problem 2, where we will run it on a small data sample. We will run the script on the big graph in Problem 4. We will find the distribution of node out-degrees to follow a power law ($1/d^k$ for some constant k and it will look roughly like a straight-line on a graph with logarithmic scales on both the x and y axes) instead of an exponential distribution. If you look at Figures 2 and 3 in [this paper](#), you will find that the degrees of web pages on the web, in general, follow a similar power law distribution. This is very interesting because it means that the Web and the semantic Web cannot be modeled as random graphs. They need a different theoretical model.

In Problem 3, we will look for paths of length 2 in a sub-graph of our big graph. This is a simple version of more complex algorithms that try to measure the diameter of a graph or try to extract other related properties. We will do all this on a very real 0.5TB graph! How cool will that look on your resume: "Analyzed properties of a 0.5TB (a billion vertices) graph using Pig/Hadoop".

You will access the following datasets in S3, through pig (using the LOAD command -- see example.pig)

```
s3n://uw-cse-344-oregon.aws.amazon.com/cse344-test-file -- 250KB. This is used in example.pig. Always use this file for debugging your scripts first!
```

```
s3n://uw-cse-344-oregon.aws.amazon.com/btc-2010-chunk-000 -- 2GB. You will use this dataset in questions 1, 2, 3..
```

```
s3n://uw-cse-344-oregon.aws.amazon.com -- 0.5TB. This directory contains 251 files btc-2010-chunk-000 to btc-2010-chunk-317 (since only 251 of the original 318 files uploaded correctly). You will use this in problem 4.
```

It is not necessary for the assignment, but if you want to inspect the files directly, you can access them over the Internet using urls of the following form (Note that accessing the 0.5TB file in this way is not recommended!):

<http://uw-cse-344-oregon.aws.amazon.com.s3.amazonaws.com/btc-2010-chunk-000>

<http://uw-cse-344-oregon.aws.amazon.com.s3.amazonaws.com/cse344-test-file>

Problem 1: Getting started with Pig on chunk-000

Modify example.pig to use the file `uw-cse-344-oregon.aws.amazon.com/btc-2010-chunk-000` instead of `uw-cse-344-oregon.aws.amazon.com/cse344-test-file`. Run on an AWS cluster with **10 small nodes**. (also see hints below).

1.1 How many MapReduce jobs are generated by example.pig?

1.2 How many reduce tasks are within the first MapReduce job? How many reduce tasks are within later MapReduce jobs?

1.3 How long does each job take? How long does the entire script take?

1.4 What is the schema of the tuples after each of the following commands in example.pig?

- After the command `ntriples = ...`
- After the command `objects = ...`
- After the command `count_by_object = ...`

Hint 1 : Use the [job tracker](#) to see the number of map and reduce tasks for your MapReduce jobs.

Hint 2: To see the schema for intermediate results, you can use Pig's interactive command line client `grunt`, which you can launch by running Pig without specifying an input script on the command line. When using `grunt`, a command that you may want to know about is [describe](#) . To see a list of other commands, type `help`.

What you need to turn in: How many records are there in `count_by_object`? DON'T FORGET TO SHUTDOWN YOUR INSTANCES!

Problem 2A: Compute a Histogram on cse344-test-file

Using the 'cse344-test-file' file , write a Pig script that groups tuples by the subject column, and creates/stores histogram data showing the distribution of counts per subject, then generate a scatter-plot of this histogram. The histogram consists of:

- The x-axis is the counts associated with the subjects, and
- The y-axis is the total number of subjects associated with each particular count.

So, for each point (x,y) that we generate, we mean to say that y subjects each had x tuples associated with them after we group by subject.

A few comments to help you get started:

- We expect that your script will (1) group the input data by subject and count the tuples associated with each subject then (2) group the results by these intermediate counts (x-axis values) and compute the final counts (y-axis values).
- To get more familiar with the Pig Latin commands, we suggest that you also take a look at the [Pig Latin Reference](#).

DEBUGGING:

- Since you are using the small test file in this question, you can run a small, **1-node cluster**.
- In this question, we are debugging the script. The output of this question is thus not going to be terribly interesting. In fact, your scatterplot should only have two points: (1,1) and (3,333).
- To debug a Pig Latin script, try to run Pig as follows:

```
pig -x local
```

Run all commands as you normally would, except for `store`. You need to store your results locally:

`store my_final_output into '/tmp/finaloutput' using PigStorage()`

- ☐ Once you are done debugging in local mode, try to run your script by issuing real MapReduce jobs. That is run with "pig" instead of "pig -x local" (remember to change the store command).

What you need to turn in: How many (x, y) points are generated in the histogram? DON'T FORGET TO SHUTDOWN YOUR INSTANCES!

Problem 2B: Compute a Histogram on chunk-000

Now run your script from Problem 2A on 'btc-2010-chunk-000' file. Please use a **5-node cluster** of small instances.

Note: this script took about 21 minutes with 5 nodes.

What you need to turn in: How many (x, y) points are generated in the histogram? DON'T FORGET TO SHUTDOWN YOUR INSTANCES!

Problem 3: Compute a Join on chunk-000

In this problem we will consider the subgraph consisting of triples whose subject matches `rdfabout.com`: for that, filter on subject matches `.*rdfabout.com.*`. Find all chains of lengths 2 in this subgraph. More precisely, return all sextuples (subject, predicate, object, subject2, predicate2, object2) where `object=subject2`.

Note: Newer versions of Pig will automatically drop the duplicate column in the join output. In that case, you do NOT need to return the sixth column.

Suggestions on how to proceed:

- First filter the data so you only have tuples whose subject matches `'rdfabout.com'`.
- Make another copy of the filtered collection (it's best to re-label the subject, predicate, and objects, for example to `subject2, predicate2, object2`).
- Now join the two copies:
 - the first copy of the `'rdfabout.com'` collection should match on `object`.
 - the second copy of the `'rdfabout.com'` collection should match on `subject2`.
- Remove duplicate tuples from the result of the join

As above, first debug your script for this problem using the test file. Once your script is debugged, then run it on the bigger file `'btc-2010-chunk-000'`. While debugging on the test file, make the following two changes:

1) Use the following filter

subject matches `.*business.*`

2) Change the join predicate to be `subject=subject2`

Otherwise, you will not get any results.

Note: this script took about 18 minutes with 10 small nodes.

What you need to turn in: How many records are generated by the join for the `cse-344-test-file` dataset? For the `btc-2010-chunk-000` dataset? DON'T FORGET TO SHUTDOWN YOUR INSTANCES!

Problem 4 : Compute a Histogram on the Entire Dataset

Compute the histogram in Problem 2 on the **entire 0.5TB dataset**. Use as many nodes as you like up to 19 small nodes.

You need to modify the load instruction to:

```
raw = LOAD 's3n://uw-cse-344-oregon.aws.amazon.com/btc-2010-chunk-\\*' USING TextLoader as (line:chararray);
```

Note: this query will take more than **4 hours to run**. Plan accordingly, and monitor carefully: if anything looks wrong, abort, fix, restart.

When you are done, appreciate how *relatively* quick and easy it was to analyze a 0.5TB graph!

What you need to turn in: How many (x, y) points are generated in the histogram? DON'T FORGET TO SHUTDOWN YOUR INSTANCES!

